

The Words You Will Hear Every Single Day

Cloud terminology, explained properly - with examples and diagrams

Session	2 - follows the introduction to cloud computing
Applies to	Both the AWS batch and the Azure batch
Contains	45 terms in 7 groups, 7 diagrams, and a one-page revision sheet
How to use it	Read the group before the topic is taught. Keep the last page for revision.
Why it exists	Most people who struggle with cloud are not struggling with the technology. They are struggling because six words in every sentence mean nothing to them.

The seven groups

Group 1	Where things live	Region, availability zone, edge location, data centre
Group 2	Running things	Instance, image, provisioning, scaling, elasticity
Group 3	Staying up	High availability, fault tolerance, disaster recovery, SLA, RTO, RPO
Group 4	Moving data	Latency, bandwidth, throughput, CDN, load balancer
Group 5	Storing data	Object, block and file storage, durability, IOPS
Group 6	Paying for it	Pay-as-you-go, CapEx, OpEx, reservations, spot, TCO
Group 7	Building and securing	API, IaC, serverless, containers, shared responsibility

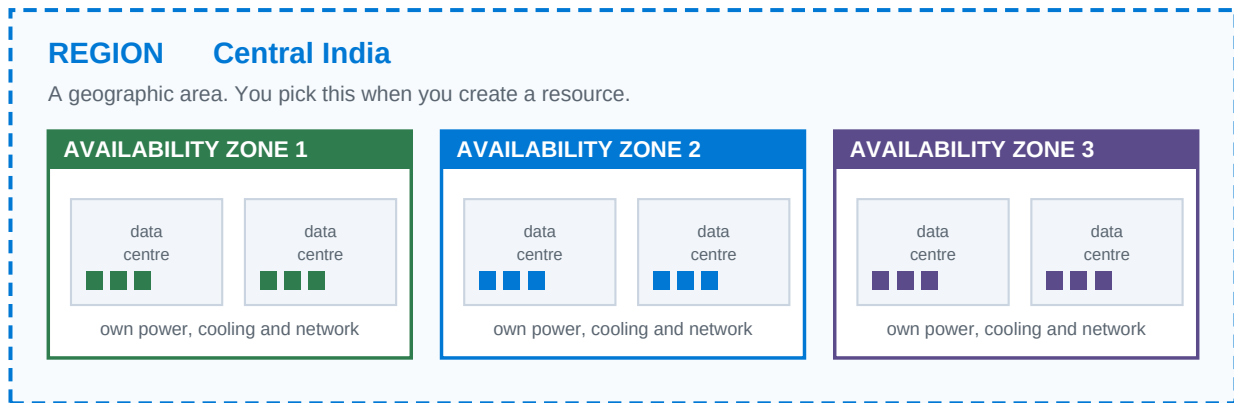
How to read a definition in this handout

Every term has four parts: what it means in plain words, a concrete example, why it matters to you, and what each provider calls it. If you only read one part, read the example.

GROUP 1

1. Where Things Live

Cloud is not in the sky. It is in buildings. These four words describe how those buildings are organised, and you will choose between them every time you create a resource.



One zone can fail completely and the other two keep running.

A region contains several availability zones. Each zone contains one or more data centres with independent power and network.

Data centre

In plain words. A physical building full of servers, with industrial power, backup generators, cooling, fire suppression and security staff.

Example. A warehouse-sized building somewhere near Pune or Chennai with thousands of racks in it. You will never see it and you will never need to.

Why it matters to you. It is the thing you are renting a slice of. Everything else in this list is a way of organising these buildings.

Region

In plain words. A geographic area where a provider operates a cluster of data centres. You choose a region when you create almost any resource.

Example. Central India, South India, Southeast Asia, West Europe. A student in Meerut normally picks Central India.

Why it matters to you. Three things depend on it. Speed - closer region means lower delay for your users. Price - the same virtual machine costs different amounts in different regions. Law - some data is legally required to stay inside the country.

AWS calls it	Region (e.g. ap-south-1)	Azure calls it	Region (e.g. Central India)
--------------	--------------------------	----------------	-----------------------------

Availability Zone

In plain words. One or more data centres inside a region that have their own power, cooling and network, physically separated from the other zones.

Example. If you run two web servers in Zone 1 and Zone 2, a flood or power failure that takes out Zone 1 leaves your site running on Zone 2.

Why it matters to you. This is the single cheapest way to make a system survive a failure. Spreading resources across zones costs nothing extra in most cases - you just have to remember to do it.

AWS calls it	Availability Zone	Azure calls it	Availability Zone
--------------	-------------------	----------------	-------------------

Edge location

In plain words. A small facility in many more cities than there are regions, used to cache content close to users. Not for running your servers.

Example. Your website is hosted in Mumbai, but a user in Delhi gets the images from a Delhi edge location instead of pulling them all the way from Mumbai.

Why it matters to you. It is how a global site feels fast everywhere without you running servers everywhere. You will meet it again under the word CDN.

AWS calls it	CloudFront edge location	Azure calls it	Azure CDN / Front Door edge
--------------	--------------------------	----------------	-----------------------------

Ek line mein

Region matlab **kaunsa sheher**, availability zone matlab **us sheher me kaunsi alag building**, aur edge location matlab **content ka chhota godam** jo user ke paas hota hai.

GROUP 2

2. Running Things

Virtual machine, or instance

In plain words. A computer made out of software. Several of them share one physical server without knowing about each other.

Example. You ask for 1 CPU and 1 GB of memory running Ubuntu. Ninety seconds later you have a machine you can log into, sitting on hardware shared with other customers.

Why it matters to you. This is the most common thing you will create in this course. Everything about sizing, pricing and scaling starts here.

AWS calls it	EC2 instance	Azure calls it	Virtual Machine
--------------	--------------	----------------	-----------------

Image

In plain words. A saved template of a disk - an operating system, and optionally your software already installed - used as the starting point for new machines.

Example. Instead of installing Ubuntu and your web server by hand on every new machine, you build it once, save an image, and every new machine starts ready to work.

Why it matters to you. It is what makes fifty identical servers possible. Without images, scaling out would mean fifty manual installations.

AWS calls it	AMI (Amazon Machine Image)	Azure calls it	VM Image / Shared Image Gallery
--------------	----------------------------	----------------	---------------------------------

Provisioning

In plain words. The act of creating a resource and having the platform make it ready for use.

Example. Clicking Create on a virtual machine and watching it reach the Running state is provisioning.

Why it matters to you. You will see this word in error messages and status fields constantly. Provisioning failed simply means the resource could not be created - and the message usually says why.

Deprovisioning, deallocating and deleting

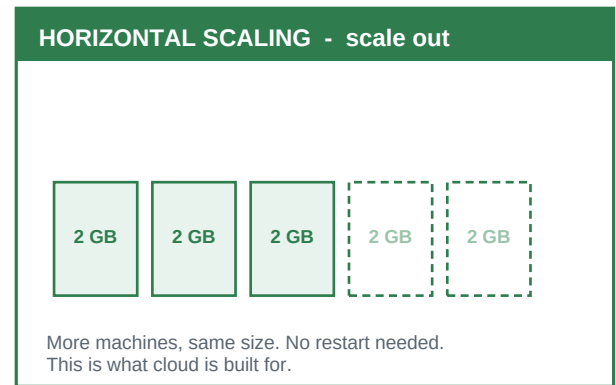
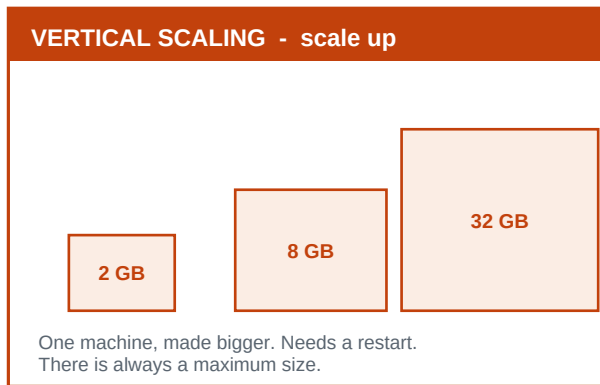
In plain words. Three different ways of stopping something, with three different bills.

Example. Stopping a VM from inside the operating system still bills you for compute. Deallocating it from the portal stops compute charges but the disk keeps costing money. Deleting removes all charges.

Why it matters to you. This distinction empties student credit faster than anything else. If you are finished with a lab resource, delete it - do not just shut it down.

Scaling - the two directions

Scaling means changing how much capacity you have. There are exactly two ways to do it, and the difference matters in every design discussion and every exam.



Scaling up makes one machine bigger. Scaling out adds more machines.

	Vertical - scale up	Horizontal - scale out
What changes	One machine gets more CPU or memory	More machines are added
Downtime	Usually needs a restart	None - new machines simply join
Limit	There is always a maximum machine size	Practically unlimited
Failure impact	One machine fails, everything is down	One machine fails, the rest carry on
Suits	Databases, legacy software that cannot be split	Web servers, APIs, stateless work
Cloud prefers	Sometimes	Almost always

Scalability

In plain words. The ability of a system to handle growth by adding resources.

Example. Your site handled 1,000 users last year and handles 50,000 this year because you added more servers along the way.

Why it matters to you. It describes a property of your design. A badly designed system cannot be scaled no matter how much money you spend.

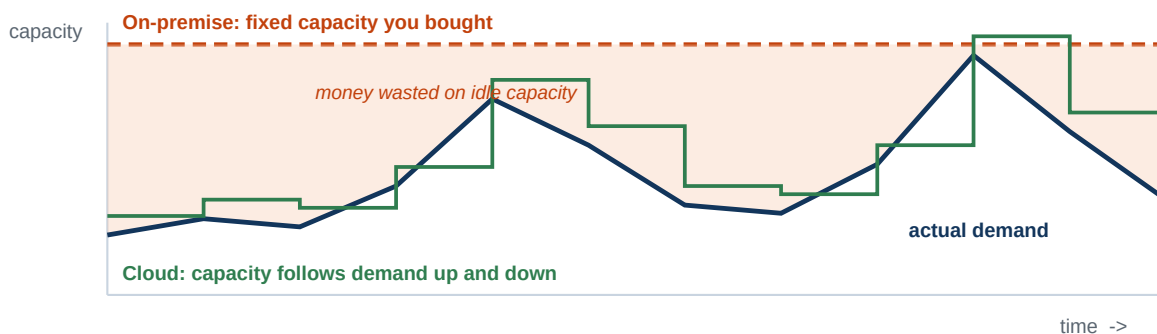
Elasticity

In plain words. Capacity that grows and shrinks automatically as demand changes, usually within minutes.

Example. Your result portal runs on 2 servers at night, automatically grows to 40 when results are announced, and shrinks back to 2 by morning. You pay for 40 servers only for those hours.

Why it matters to you. Elasticity is the single biggest financial argument for cloud. Scalability says you can grow; elasticity says you also shrink, and stop paying.

AWS calls it	Auto Scaling Group	Azure calls it	Virtual Machine Scale Set
--------------	--------------------	----------------	---------------------------



The orange shaded area is money spent on capacity nobody used. Cloud removes it by letting capacity follow demand.

Scalability and elasticity are not the same word

Students use them interchangeably and lose marks for it. Scalability is the ability to grow. Elasticity is growing and shrinking automatically, in response to real demand, without a human deciding. A system can be scalable without being elastic.

Ek line mein

Scalability matlab **bada ho sakta hai**. Elasticity matlab **apne aap bada aur chhota dono hota hai** - aur chhota hone pe paisa bachta hai.

Auto scaling

In plain words. A rule that adds or removes machines automatically based on a measurement you choose.

Example. A rule that says: if average CPU stays above 70 per cent for five minutes, add a server; if it drops below 30 per cent, remove one. Minimum 2, maximum 10.

Why it matters to you. It is elasticity made real. Without a rule, nothing scales by itself and someone has to watch a dashboard at two in the morning.

AWS calls it	Auto Scaling	Azure calls it	Autoscale / Scale Sets
--------------	--------------	----------------	------------------------

GROUP 3

3. Staying Up

This group is where beginners are most often imprecise, and where the exam is most precise. Read this section slowly.

High availability

In plain words. Designing a system so that it keeps working when a component fails - though there may be a short interruption while it recovers.

Example. Two web servers behind a load balancer. One dies. Users may see a two-second hiccup, then everything continues on the surviving server.

Why it matters to you. This is the realistic target for most systems, and it is what spreading across availability zones gives you.

Fault tolerance

In plain words. Designing a system so that a failure causes no interruption at all - not even a brief one.

Example. An aircraft control system, or a payment switch, where a component failing must be invisible to the user.

Why it matters to you. Fault tolerance is significantly more expensive than high availability because it needs full duplication running at all times. Most business systems do not need it, and being able to say why is the mark of someone who understands the trade-off.

Disaster recovery

In plain words. The plan and the resources for bringing a system back after something large fails - an entire region, a flood, a serious human error.

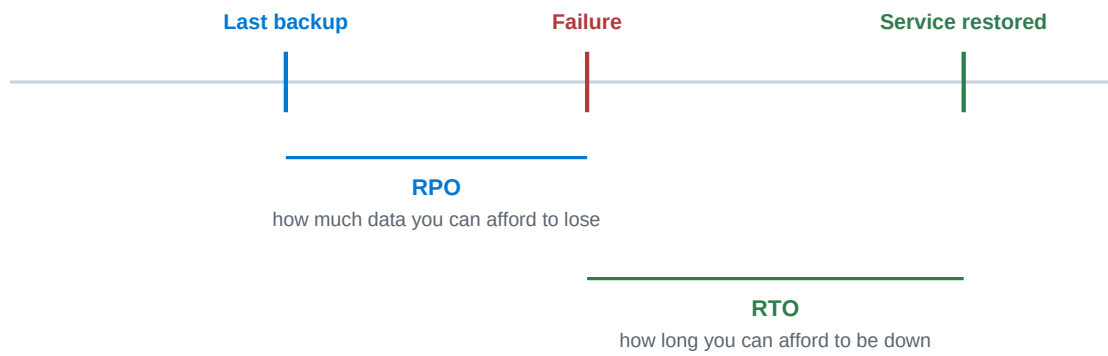
Example. Your primary system runs in Central India. Backups replicate to Southeast Asia. If Central India becomes unavailable, you bring the system up there.

Why it matters to you. High availability handles a machine failing. Disaster recovery handles the whole site failing. They are different problems and need different plans.

	High availability	Fault tolerance	Disaster recovery
Protects against	A component failing	A component failing	A whole site or region failing
Interruption	Brief	None	Planned, measured in RTO
Cost	Moderate	High	Varies with how fast you need to recover
Typical approach	Multiple zones	Full duplication, running	Replication to another region

RTO and RPO - the two numbers that define a recovery plan

Every disaster recovery conversation eventually becomes a conversation about these two numbers. They sit on opposite sides of the moment of failure.



RPO looks backwards from the failure - how much data is lost. RTO looks forwards - how long until service returns.

RPO - Recovery Point Objective

In plain words. How much data you can afford to lose, measured in time. It is decided by how often you take backups.

Example. If you back up every 6 hours and the system fails 5 hours after the last backup, you have lost 5 hours of data. An RPO of 6 hours means that is acceptable.

Why it matters to you. A bank cannot accept losing 6 hours of transactions, so it replicates continuously. A college photo gallery probably can. The answer drives the cost.

RTO - Recovery Time Objective

In plain words. How long the system may be down before service must be restored.

Example. An RTO of 4 hours means that after a failure, the business accepts up to 4 hours of outage and no more.

Why it matters to you. A shorter RTO means more infrastructure kept ready, which means more money. This is the trade-off you will be asked to justify.

SLA - Service Level Agreement

In plain words. The provider's formal, written commitment about uptime, and what you are owed if they miss it.

Example. A commitment of 99.9 per cent monthly uptime allows roughly 43 minutes of downtime in a 30-day month. 99.99 per cent allows about 4 minutes.

Why it matters to you. Two points matter. First, adding a nine costs real money. Second, an SLA usually pays you a service credit, not the money your business actually lost - so the SLA is not insurance.

Uptime commitment	Downtime allowed per month	Downtime allowed per year
99%	about 7 hours 18 minutes	about 3 days 15 hours
99.9%	about 43 minutes	about 8 hours 45 minutes
99.95%	about 21 minutes	about 4 hours 22 minutes
99.99%	about 4 minutes 20 seconds	about 52 minutes

Ek line mein

RPO = kitna data kho sakte ho. RTO = kitni der band reh sakte ho. SLA = provider ka likhit vaada. Ye teeno numbers hain, feelings nahi.

Redundancy

In plain words. Keeping more than one of something so that losing one does not stop the service.

Example. Three copies of every file, two web servers instead of one, two network paths instead of one.

Why it matters to you. Redundancy is the mechanism. High availability and fault tolerance are the results you get from applying it.

Failover

In plain words. The automatic switch from a failed component to a standby one.

Example. The primary database stops responding, and within seconds the standby is promoted and starts serving traffic.

Why it matters to you. Failover that has never been tested is not failover, it is hope. Testing it deliberately is a normal part of operations.

4. Moving Data

Latency

In plain words. The delay between sending a request and getting the first response, measured in milliseconds. It is mostly decided by distance.

Example. A request from Meerut to a server in Mumbai might take 30 ms. The same request to a server in the United States might take 250 ms - and a page that makes twenty such requests feels noticeably slow.

Why it matters to you. This is why choosing the nearest region matters. No amount of money makes data travel faster than the speed of light through fibre.

Bandwidth and throughput

In plain words. Bandwidth is how much data the connection could carry. Throughput is how much it actually carries.

Example. A pipe with a wide mouth is bandwidth. The water actually flowing through it is throughput. A blockage or a slow tap keeps throughput below bandwidth.

Why it matters to you. People blame bandwidth for slowness when the real problem is latency. A wide pipe with a long delay still feels slow for small, frequent requests.

Latency and bandwidth are different problems

Think of a truck full of hard drives driving from Delhi to Mumbai. Enormous throughput - terabytes delivered. Terrible latency - it takes a day. Now think of a phone call: tiny throughput, but the delay is milliseconds. Video streaming needs bandwidth. Online gaming needs low latency. Knowing which one a workload needs is the skill.

CDN - Content Delivery Network

In plain words. A network of edge locations that keeps copies of your static content close to users.

Example. Your site is hosted in Mumbai. A user in Chennai gets the logo, stylesheets and images from a Chennai edge cache instead of from Mumbai. Only the parts that must be fresh come from the origin.

Why it matters to you. It reduces latency for users and reduces load on your servers at the same time. It only helps content that does not change per user.

AWS calls it	CloudFront	Azure calls it	Azure CDN / Front Door
--------------	------------	----------------	------------------------

Load balancer

In plain words. A device that receives incoming traffic and spreads it across several backend servers.

Example. One address that users connect to. Behind it, five web servers. If one stops responding, the load balancer stops sending traffic to it and users notice nothing.

Why it matters to you. It is what makes horizontal scaling usable. Without it, users would have to know which server to connect to.

AWS calls it	Elastic Load Balancer	Azure calls it	Load Balancer / Application Gateway
--------------	-----------------------	----------------	-------------------------------------

Ingress and egress

In plain words. Data coming into the cloud, and data going out of it.

Example. Uploading a 10 GB backup to cloud storage is ingress. Your users downloading 10 GB of video is egress.

Why it matters to you. Providers usually charge little or nothing for ingress and charge real money for egress. Large surprise bills are very often egress bills.

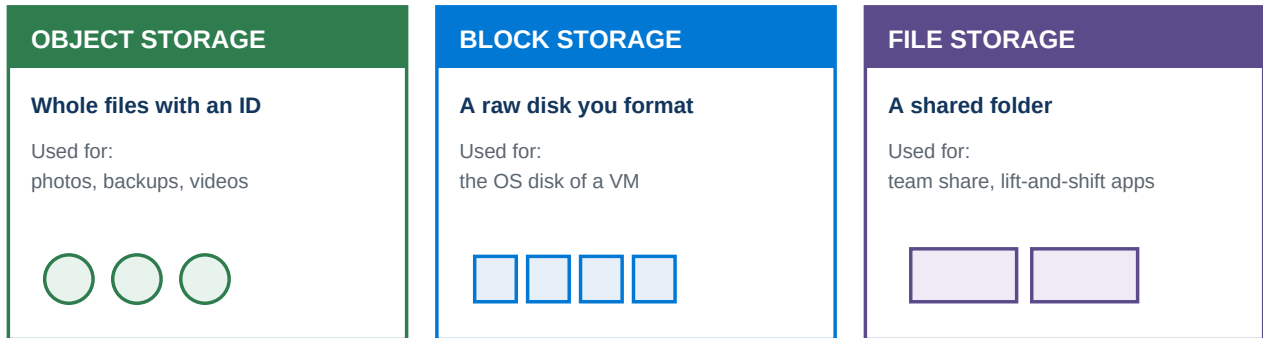
Ek line mein

Latency = **kitni der lagti hai**. Bandwidth = **kitna chauda rasta hai**. CDN = **content ko user ke paas rakh do**. Egress = **bahar bhejne ka paisa lagta hai**.

GROUP 5

5. Storing Data

There are three ways to store data in the cloud, and they are not interchangeable. Choosing the wrong one is a common and expensive beginner mistake.



Three storage types. The question to ask is how the data will be accessed, not how big it is.

Object storage

In plain words. Storage that holds whole files as objects, each with an identifier and its own metadata. You fetch a whole object; you do not edit part of it in place.

Example. Photographs, video files, backups, log archives, the static files of a website.

Why it matters to you. It is cheap, effectively unlimited, and reachable over the internet. It is the default answer for large amounts of data that is written once and read many times. It is a poor choice for anything that needs constant small edits.

AWS calls it	S3 bucket	Azure calls it	Blob Storage (inside a Storage Account)
--------------	-----------	----------------	---

Block storage

In plain words. A raw virtual disk attached to one machine. The operating system formats it and treats it exactly like a physical disk.

Example. The disk your virtual machine boots from, or a second disk you attach to hold a database.

Why it matters to you. It is fast and supports small, frequent writes, which is what databases and operating systems need. It is normally attached to one machine at a time.

AWS calls it	EBS volume	Azure calls it	Managed Disk
--------------	------------	----------------	--------------

File storage

In plain words. A shared folder that several machines can mount at the same time, using normal file protocols.

Example. A shared directory that five application servers all read configuration and uploads from.

Why it matters to you. It is the easiest way to move an older application to the cloud when that application expects a network drive to exist.

AWS calls it	EFS / FSx	Azure calls it	Azure Files
--------------	-----------	----------------	-------------

Durability

In plain words. The probability that data you stored is still intact after a period of time.

Example. A commitment often written as eleven nines. In practical terms, the provider keeps multiple copies across separate locations so that hardware failure does not lose your data.

Why it matters to you. Durability is about not losing data. Availability is about being able to reach it right now. A service can be extremely durable and still be briefly unavailable.

IOPS and redundancy tiers

In plain words. IOPS counts how many read or write operations a disk can perform each second. Redundancy tiers describe how many copies of your data are kept and where.

Example. A basic disk might handle a few hundred IOPS; a premium disk far more. A locally redundant tier keeps copies in one data centre, a geo-redundant tier keeps copies in another region.

Why it matters to you. Database performance is usually limited by IOPS, not by CPU. And the redundancy tier you pick is a direct trade between cost and how big a disaster you can survive.

Ek line mein

Photo aur backup = object storage. VM ki disk = block storage. Shared folder = file storage. Ye teen alag cheezein hain, ek dusre ki jagah nahi chalti.

6. Paying For It

Pay-as-you-go

In plain words. You are billed for what you actually consume, measured in small units, with no commitment.

Example. A virtual machine billed by the second it runs. Storage billed by the gigabyte per month. Delete the resource and the charge stops.

Why it matters to you. It is the default for everything you create in this course. It is also the most expensive per hour - you pay a premium for having made no commitment.

CapEx and OpEx

In plain words. Capital expenditure is a large one-time purchase. Operating expenditure is an ongoing running cost.

Example. Buying a server for a few lakh rupees is CapEx. Renting the equivalent capacity monthly is OpEx.

Why it matters to you. This is why finance departments care about cloud. Moving from CapEx to OpEx changes how a business plans its money, not just how it runs its computers.

Reserved capacity

In plain words. Committing to use a resource for one or three years in exchange for a large discount.

Example. A virtual machine that runs continuously all year, reserved for three years at a substantially lower rate than on-demand.

Why it matters to you. The catch is the commitment. You pay whether or not you use it. It suits steady, predictable workloads and is wrong for anything experimental.

AWS calls it	Reserved Instances / Savings Plans	Azure calls it	Reservations / Savings Plans
--------------	------------------------------------	----------------	------------------------------

Spot capacity

In plain words. Spare capacity offered very cheaply, on the condition that the provider may take it back with little warning.

Example. Batch processing, video rendering, or test workloads that can be interrupted and restarted without anyone caring.

Why it matters to you. Very large savings, but only for work that tolerates being interrupted. Never put a database or a customer-facing website on it.

AWS calls it	Spot Instances	Azure calls it	Spot Virtual Machines
--------------	----------------	----------------	-----------------------

Pricing model	Discount	Commitment	Right for
Pay-as-you-go	None	None	Learning, testing, unpredictable workloads
Reserved	Large	1 or 3 years	Steady workloads that run continuously
Spot	Very large	None, but can be taken back	Interruptible batch work

TCO - Total Cost of Ownership

In plain words. The real total cost of running something, including the parts people forget.

Example. A server does not cost only its purchase price. Add electricity, cooling, rack space, backup hardware, the administrator's salary and the cost of the downtime when it fails.

Why it matters to you. TCO is the honest way to compare cloud against owning hardware. Comparing only the sticker prices always produces the wrong answer.

Cost optimisation

In plain words. The ongoing practice of paying only for what you actually need.

Example. Deleting forgotten test machines, resizing an oversized VM, moving old backups to a cheaper storage tier, buying reservations for the servers that never switch off.

Why it matters to you. Cloud bills grow quietly. The most common cause is not expensive services - it is resources nobody remembered to delete.

The habit to build now

Every lab in this course ends with deleting what you created. That is not just about protecting your free credit. It is the single most valuable operational habit in this entire subject, and the reason most real cloud bills are higher than they need to be.

7. Building and Securing

API - Application Programming Interface

In plain words. A defined way for one program to ask another program to do something, without a human clicking anything.

Example. The portal is a website that calls the cloud provider's API. When you run a CLI command, it calls the same API. Your own code can call it too.

Why it matters to you. This is why cloud can be automated at all. Everything you can do by clicking, you can do by calling an API - which means it can be scripted, repeated and reviewed.

Infrastructure as Code

In plain words. Describing your infrastructure in a text file and letting a tool build it, instead of clicking through a portal.

Example. A file that declares one network, three subnets and two virtual machines. Run it and you get exactly that. Run it in a second region and you get an identical copy.

Why it matters to you. It makes environments repeatable and reviewable. It also means your infrastructure has a history, because the file lives in version control alongside your code.

AWS calls it	CloudFormation / CDK	Azure calls it	ARM Templates / Bicep
--------------	----------------------	----------------	-----------------------

Managed service

In plain words. A service where the provider runs the software for you - installation, patching, backups and availability included.

Example. Running MySQL yourself on a virtual machine means you patch it, back it up and fix it at 3 a.m. A managed database service does those for you.

Why it matters to you. You trade some control for a large reduction in work. Most modern architectures use managed services wherever they can.

Serverless

In plain words. A model where you supply code and the provider runs it on demand, with no server for you to manage and no charge when it is idle.

Example. A function that runs whenever a file is uploaded, resizes the image and exits. If nothing is uploaded for a week, it costs nothing.

Why it matters to you. The name is misleading - servers obviously still exist. What is true is that you never see them, never patch them, and never pay for them sitting idle.

AWS calls it	Lambda	Azure calls it	Azure Functions
--------------	--------	----------------	-----------------

Container

In plain words. A package holding an application together with everything it needs to run, isolated from whatever else is on the machine.

Example. The same container image runs identically on your laptop, on a test server and in production. The familiar excuse that it works on my machine stops being available.

Why it matters to you. Containers start in seconds and are far lighter than virtual machines, which is why they suit horizontal scaling so well.

Orchestration

In plain words. Software that decides where containers run, restarts them when they fail, and scales them up and down.

Example. Kubernetes keeping ten copies of your application running across a group of machines, replacing any that crash.

Why it matters to you. Running one container is easy. Running hundreds reliably is the problem orchestration exists to solve.

AWS calls it	ECS / EKS	Azure calls it	AKS
--------------	-----------	----------------	-----

Security terms you will meet immediately

Shared responsibility model

In plain words. An agreed division of security work. The provider secures the cloud itself. You secure what you put into it.

Example. Microsoft or Amazon guards the building and patches the virtualization layer. You choose whether your database is exposed to the internet, and who has access to your data.

Why it matters to you. Most cloud security incidents are not the provider failing. They are a customer leaving storage public or a password weak. Knowing which side of the line a task falls on is the whole point of this model.

	ON-PREMISE	IaaS	PaaS	SaaS
Data and access	YOU	YOU	YOU	YOU
Application	YOU	YOU	YOU	PROVIDER
Runtime	YOU	YOU	PROVIDER	PROVIDER
Operating system	YOU	YOU	PROVIDER	PROVIDER
Virtualization	YOU	PROVIDER	PROVIDER	PROVIDER
Servers and storage	YOU	PROVIDER	PROVIDER	PROVIDER
Network and facility	YOU	PROVIDER	PROVIDER	PROVIDER

Notice the top row is always yours. Your data and who can reach it is never the provider's job.

As you move from left to right the provider takes on more. Your data and who can access it never moves.

Identity and access management

In plain words. The system that decides who can sign in and what they are allowed to do.

Example. A junior engineer who may read logs but may not delete a database. A billing person who may see costs but may not create resources.

Why it matters to you. Everything you do in the cloud passes through this. It is also the most common place for a costly mistake - giving everyone full access because it is easier.

AWS calls it	IAM users, roles and policies	Azure calls it	Microsoft Entra ID with RBAC
--------------	-------------------------------	----------------	------------------------------

Principle of least privilege

In plain words. Give every person and every program the minimum access needed to do their job, and nothing more.

Example. A backup script needs to read files and write to one storage location. It does not need permission to delete virtual machines.

Why it matters to you. When an account is compromised, least privilege is what limits the damage to a small area instead of the whole environment.

Encryption at rest and in transit

In plain words. At rest means data is encrypted while stored on disk. In transit means it is encrypted while travelling across a network.

Example. Your storage account encrypts files on disk - that is at rest. Your browser uses HTTPS to reach the site - that is in transit.

Why it matters to you. Both are needed, and they protect against different things. Encryption at rest does nothing for data being intercepted on the wire, and the reverse is equally true.

Multi-factor authentication

In plain words. Requiring a second proof of identity in addition to a password.

Example. A password plus a code from an app on your phone. Someone who steals the password still cannot sign in.

Why it matters to you. It is the single most effective security control available to you, and it takes two minutes to enable. Turn it on for your own cloud account today.

Terms about moving and being locked in

Multi-tenancy

In plain words. Several customers sharing the same physical hardware, kept isolated from each other by software.

Example. Your virtual machine and a company in another country may be running on the same physical server, with no way to see each other.

Why it matters to you. It is what makes cloud affordable. It is also why some regulated organisations pay extra for dedicated hardware.

Migration and lift-and-shift

In plain words. Migration is moving systems to the cloud. Lift-and-shift is the simplest form of it - moving an application as it is, without redesigning it.

Example. Copying an existing Windows application onto a cloud virtual machine, unchanged, and pointing users at the new address.

Why it matters to you. Lift-and-shift is fast and low-risk, but you keep the old design and gain few cloud benefits. It is often the right first step, rarely the right last one.

Vendor lock-in

In plain words. Becoming dependent on one provider's proprietary services, so that leaving becomes expensive and slow.

Example. An application built entirely around one provider's serverless and database services cannot be moved elsewhere without being substantially rewritten.

Why it matters to you. It is a trade-off, not a sin. Proprietary managed services genuinely save enormous effort. The mistake is walking into the trade blindly rather than choosing it deliberately.

Hybrid cloud

In plain words. Running part of your systems in your own data centre and part in the public cloud, connected together.

Example. A bank keeps its core records on its own hardware for regulatory reasons, while running its customer-facing website in the public cloud.

Why it matters to you. Almost every large organisation is hybrid in practice. It brings the complexity of both worlds, and the networking module is where that complexity becomes visible.

8. The Twelve Terms Students Most Often Confuse

These two	Are confused because	The actual difference
Scalability / Elasticity	Both are about growing	Elasticity also shrinks, automatically, and saves money
High availability / Fault tolerance	Both are about staying up	Fault tolerance allows no interruption at all, and costs far more
High availability / Disaster recovery	Both involve backups	HA handles a component failing; DR handles a whole site failing

These two	Are confused because	The actual difference
RTO / RPO	Both are recovery numbers	RTO is time to recover; RPO is data you can lose
Durability / Availability	Both sound like reliability	Durability is not losing data; availability is reaching it now
Region / Availability Zone	Both are locations	A region contains several zones; zones fail independently
Latency / Bandwidth	Both are about network speed	Latency is delay; bandwidth is capacity
Object / Block storage	Both store data	Object holds whole files; block is a disk you format
Vertical / Horizontal scaling	Both add capacity	Vertical enlarges one machine; horizontal adds machines
Stopped / Deallocated / Deleted	All sound like off	Only deleted stops every charge
CapEx / OpEx	Both are money	CapEx is one large purchase; OpEx is ongoing running cost
Ingress / Egress	Both are data transfer	Egress is what you are usually billed for

9. Test Yourself

Answer in one sentence each, without looking back.

1. What is the difference between a region and an availability zone?
2. Your system must never be interrupted, not even for a second. Which term describes that requirement?
3. You back up every 4 hours. What does that make your RPO?
4. An SLA promises 99.9 per cent uptime. Roughly how much downtime is allowed per month?
5. Which scaling direction usually requires a restart, and why?
6. Give one workload that suits spot capacity and one that does not.
7. Where would you store a large archive of exam result PDFs, and which storage type is it?
8. Under the shared responsibility model, who is responsible for a storage container being publicly readable?
9. Explain the difference between latency and bandwidth to someone using one example.
10. Your monthly bill has grown and nobody changed anything. What is the most likely cause?
11. Why is elasticity a financial argument and not only a technical one?
12. What is the difference between deallocating a virtual machine and deleting it?

Answers

1. A region is a geographic area; an availability zone is one of several independently powered locations inside that region.
2. Fault tolerance.
3. Up to 4 hours of data loss.
4. About 43 minutes.
5. Vertical, because the machine must be resized and restarted to use the new CPU or memory.
6. Suits: batch rendering or test workloads. Does not suit: a production database or a customer-facing website.
7. Object storage - the files are written once and read many times, and are never edited in place.
8. You are. The provider secures the platform; configuration of your resources is your side of the line.
9. A reasonable answer: a truck carrying hard drives has huge bandwidth but terrible latency; a phone call has tiny bandwidth but almost no delay.
10. Resources that were created and never deleted.
11. Because capacity shrinking means the bill shrinks too - you stop paying for what you are not using.
12. Deallocating stops compute charges but the disk still costs money. Deleting removes every charge.

10. One-Page Revision Sheet

Print this page. It is the fastest revision for a viva or an exam.

Places

Data centre	A building full of servers
Region	A geographic area containing several zones. You pick it per resource.
Availability Zone	An independently powered location inside a region
Edge location	A small cache near users, for content only

Staying up

High availability	Survives a component failing, with a brief interruption
Fault tolerance	Survives a component failing with no interruption. Expensive.
Disaster recovery	Survives a whole site failing. Measured by RTO and RPO.
RTO	How long you may be down
RPO	How much data you may lose
SLA	The provider's written uptime commitment. 99.9% is about 43 min a month.

Growing

Vertical scaling	One machine, made bigger. Needs a restart. Has a ceiling.
Horizontal scaling	More machines. No restart. What cloud prefers.
Scalability	Able to grow
Elasticity	Grows and shrinks automatically with demand

Storage

Object	Whole files with an ID. Photos, backups, archives.
Block	A raw disk attached to one machine. VM disks, databases.
File	A shared folder several machines mount.
Durability	Not losing the data
Availability	Being able to reach it right now

Money

Pay-as-you-go	No commitment, highest rate
Reserved	1 or 3 year commitment, large discount, steady workloads
Spot	Cheapest, can be taken back, interruptible work only
CapEx / OpEx	One large purchase versus ongoing running cost
Egress	Data leaving the cloud. This is the part you get billed for.

Security

Shared responsibility	Provider secures the cloud; you secure what you put in it
Least privilege	Minimum access needed, nothing more
At rest / in transit	Encrypted on disk versus encrypted while travelling
MFA	Password plus a second proof. Enable it today.

If you can explain fifteen of these to a friend without notes, you are ready for the next session.